

CIS 5820 Full-Stack Development Project

A Self-Directed Graduate Studio Mobile Application Project

This document covers everything about the project. Use it as your reference all term.

Contents

Foreword

1. Your Mission
2. How This Works
3. Project Guidelines
4. Mobile App Categories and Ideas
5. What Goes Into a Mobile App: The Capability Map
6. Schedule
7. What Progress Looks Like
8. The Midpoint Video
9. The Eleven Specifications
10. How Your Project Grade Is Determined
11. Tokens
12. Final Submission and Presentation
13. AI and Outside Resources

Foreword

This project is the center of this course. Everything else supports what happens here: you will design, plan, and build a mobile application of your own choosing, and you will do it the way a self-directed professional does, with real deliverables on a real schedule.

That is a deliberate departure from a purely theoretical graduate course. You have already experienced coursework in other classes, and in most cases a capstone, so the technical bar is not what makes this different. What is different is the expectation that you manage yourself: you set the direction, and you are accountable for showing real, visible progress the entire way through, not just at the end.

I am still your manager on this, and that means two things. First, it is genuinely yours: you choose what to build, and you should enjoy building it. There is real room here for exploration and for learning something you did not already know. Second, a manager who never hears from you is not managing, and a professional who never reports status is not being self-directed, they are just quiet. Reporting real, visible progress on a schedule is not a constraint on your autonomy here. It is what autonomy looks like on an actual team.

None of this means the project has to be flawless, or even finished. Parts of it may not work by the end, and that is fine. What is not fine is going quiet, producing nothing across a reporting period, or submitting a log with no evidence behind it. If you are stuck, tell me. If the scope needs to change, tell me, and we will adjust it together. The one thing this project does not tolerate is silence.

1. Your Mission

Build a mobile application you actually want to build. This is your project to define: choose something you have wanted to make, something you have seen and want your own version of, or something you think solves a real problem. Imitation is a legitimate way to learn, and rebuilding something you admire will teach you more than inventing something you do not care about.

The one firm requirement is that it must be a mobile application. Beyond that, the subject is yours, and it should be something you want to spend a semester on.

Build with Android Studio, using whatever it gives you to work with. Kotlin is the modern default; Java also works. Use the tools the environment hands you.

2. How This Works

You are the developer, and this is your project to design, plan, and manage, including how you will learn what you need for it. I am your manager. You are expected to operate with minimal day-to-day direction: you decide what you are building and how, and you are responsible for planning a schedule you can actually meet.

A working relationship still has obligations attached, regardless of how senior or self-directed the person doing the work is. You keep your manager informed, you deliver on the dates you committed to in your own proposal, and you show real, demonstrable work rather than descriptions of work you intend to do.

Your project grade is determined by specifications. Each requirement is either met or not met on a stated date, and your grade follows from which ones you complete. The quality of your app does not lower your grade. Failing to do the work, or failing to show that you did it, does.

Layoffs are real. Failure to meet the requirements of this project can result in being laid off, which means a course grade of F. See the section Professionalism and Full-Stack Projects in the syllabus for the full range of conduct that can lead to a layoff. It extends well beyond this project.

3. Project Guidelines

- Your project must be a mobile application, built in Android Studio.
- Use languages and tools Android Studio provides. Kotlin is recommended; Java is fine.
- Your proposal is your plan. Complete it fully, including the schedule and capability map, since your project is measured against what you proposed.
- Learning a new tool, framework, or technique is legitimate project work, not a delay from the real work. If you need to learn something to build your project, plan it into your schedule as its own milestone.
- Do your own work, but take advantage of tools that exist. You may use development tools, generative AI, tutorials, and references, provided you document what you used and explain your role in the result.
- All project materials go in the shared class repository, in your own folder. See The Shared Class Repository below for how this works.
- Keep every AI conversation from the start of the project. These are artifacts of your work and are submitted with your logs and final submission.
- If you want to pursue an idea that does not fit these guidelines, talk to me first.

- Ask me questions directly. Other instructors and advisors will not know the expectations for this project.

The Shared Class Repository

This project uses one repository for the whole class, with a folder for each student. Your folder is named with your last name followed by your first name, with no space, for example smithjohn. You commit your work to your own folder. This is the open-source approach described in the syllabus, where a shared, visible repository is the normal way software gets built.

Only add, edit, or delete files inside your own folder. Do not touch another student's folder or anything at the top level without talking to me first. Your classmates will be able to see your folder, and you will be able to see theirs. That is intentional.

Everything you produce for this project goes in your folder, with one exception: your video. Videos do not belong in the repository. Host it on OneDrive, YouTube, or a similar service, and share the link.

4. Mobile App Categories and Ideas

You are proposing your own app. What follows is a set of categories to help you figure out what kind of thing you want to build.

The examples under each category are illustrations of what apps in that category tend to look like. They are not a list to pick from, though there is nothing wrong with being inspired by one. Most apps fall into one of these categories, many span two or three, and a genuinely original idea may fit none of them. On your proposal you will name the categories your app falls into, or describe your own.

It Does Not Have to Be Original

Nobody expects you to invent an app that has never existed. Build a version of an app you have seen and liked, rebuild a tool you already use, or make your own take on something you enjoy. What makes it yours is the building and the choices you make along the way, not the novelty of the concept.

If Nothing Here Fits

If your idea does not fit any of these categories, describe it in your own words on the proposal and we will talk about it. Original ideas are welcome. It still has to be a mobile app.

Choosing and Sizing

Pick something you would be a little annoyed to abandon. Motivation is the resource that runs out first, and interest is what keeps a semester project alive in week ten.

A mobile app can balloon fast. If an idea feels too large, narrow it to a few screens done properly rather than a dozen half-built. If it feels too small, add a dimension such as saving data on the device, pulling data from the internet, or using a device feature like the camera or location. We will right-size it together when I review your proposal.

Table 1. Mobile app categories and ideas

A List of Categories and Ideas

Utility and Productivity

Apps that help someone get something done.

- To-do list or task manager with reminders
- Habit or streak tracker
- Budget or expense tracker
- Note-taking or journaling app
- Unit or currency converter with interface
- Countdown or event timer for something you care about

Information and Reference

Apps that organize or surface information.

- Recipe browser with search and favorites
- Study or flashcard app
- Local guide for restaurants, trails, or events
- Plant, animal, or product identifier
- Reference app for a domain you know well
- Personal catalog for books, media, or a collection

Health and Fitness

Apps that track or encourage a habit or activity.

- Workout logger with history
- Water, sleep, or step tracker
- Meditation or breathing timer
- Meal or calorie logger
- Stretching or exercise guide
- Mood tracker with trends over time

Device-Powered

Apps whose whole point is using what the phone can do.

- Camera app with a twist (scanning, overlays, filters)
- Location-based reminder or check-in app
- Compass, level, or measurement tool using sensors
- Step or motion detector
- Flashlight or utility toolkit
- Augmented-reality or map-based

Games

Something to play. Simple is fine; polished-and-small beats ambitious-and-broken.

- Trivia or quiz game with categories
- Card or dice game
- Tile, match, or puzzle game
- Memory or reaction game
- Text-based adventure with choices
- Idle or clicker game with saved progress

Media and Creativity

Apps built around sound, images, or making things.

- Music or sound-effect player
- Photo gallery with filters or tagging
- Drawing or sketch pad
- Meme or image caption maker
- Simple video or audio recorder
- Color palette or design tool

Social and Community

Apps built around people, sharing, or groups. Keep the scope realistic; a full social network is too much, but a slice of one is not.

- Shared shopping or packing list
- Event or meetup planner for groups
- Local message board or bulletin
- Photo-sharing prototype for a group
- Poll or voting app for a group
- Study-group or team coordinator

5. What Goes Into a Mobile App: The Capability Map

With your proposal you complete a map of the capabilities your app will use. The purpose is not coverage. The purpose is to make you think about what actually goes into a mobile app before you start building it, and to notice the pieces your app will need.

This is not a checklist. You are not expected to use every capability. Nobody is counting. Mark what your app will genuinely use and leave the rest alone.

Table 2. Mobile app capabilities to consider

Capability	What it means, with examples
Screens and Navigation	Multiple screens and moving between them. Example: a home screen, a detail screen, a settings screen, and the buttons or gestures that move between them.
User Input and Forms	Taking input from the user. Example: text fields, buttons, switches, date pickers, and checking that what they entered makes sense before you use it.
Displaying Lists of Data	Showing collections of items. Example: a scrolling list of tasks, a grid of photos, cards the user can tap to open.
Local Data Storage	Saving data on the device so it survives closing the app. Example: saved notes, a to-do list that is still there tomorrow, user settings that stick.
Networking and Outside Data	Getting data from the internet. Example: calling a public API for weather, sports scores, or recipes, and showing the results in your app.
Device Features	Using the hardware. Example: the camera, photo library, GPS and location, the microphone, or motion sensors.
Notifications and Alerts	Getting the user's attention. Example: a reminder notification, an alarm, an in-app alert when something needs a response.
Media	Sound, images, and video. Example: playing a sound effect, showing images, embedding a video, recording audio.
App State and Lifecycle	Handling how a mobile app actually runs. Example: remembering where the user was, surviving a screen rotation, saving progress when the app goes to the background.
Visual Design and Theming	Making it look intentional. Example: consistent colors and spacing, a light and dark theme, icons, a layout that works on different screen sizes.
Permissions	Asking the user for access. Example: requesting permission to use the camera, location, or notifications, and handling it gracefully if

Capability	What it means, with examples
	they say no.
Other	Anything your app needs that is not listed above, such as maps, authentication and login, in-app purchases, sharing to other apps, or a background service.

You do not have to know at proposal time exactly how each piece will work. A reasonable plan is enough, and the map can change as the app develops. You update it at the end with what you actually used.

6. Schedule

These dates are fixed.

Table 3. *Project schedule*

Week	What Is Due	Format
2	Proposal	Proposal with schedule, capability map, and other sections
4	Log 1	Log with artifact
6	Log 2	Log with artifact
7	Midpoint video	10 to 15 minute technical demonstration, recorded or live
9	Log 3	Log with artifact
11	Log 4	Log with artifact
13	Log 5	Log with artifact
16	Log 6	Log with artifact
17	Final submission	Deliverable and repository, final video, documentation form

See the Tentative Schedule in Course Information for specific due dates.

Logs use the log template provided with the project materials. Use the template; do not invent your own format.

7. What Progress Looks Like

Progress is not the same as finished features. On a project you designed yourself, progress is whatever moved you closer to being able to build the thing, and that varies depending on what you ran into.

Every log must include an artifact

A log is not just a statement that you worked on the project. Every log must include something that shows the work rather than describes it. Examples of acceptable artifacts:

- Screenshots
- Source code
- A running prototype you can demonstrate
- A completed learning module, tutorial, or short course
- Exercises completed from a tutorial
- Design documents or sketches
- Testing results
- AI chat logs
- Anything else that clearly shows work happened

The artifact is not graded on quality or polish. A rough screenshot or a messy sketch is enough. What it cannot be is nothing. A log with words and no artifact does not meet the specification, the same as a log that was never submitted.

Progress includes

- Learning a tool, framework, or part of Android you needed. If your proposal scheduled learning time and you spent it learning, that is progress, and the artifact is whatever you produced along the way: notes, a completed tutorial, a small test program.
- Trying an approach that failed and understanding why. A dead end you can explain, with the code or notes to show for it, is worth more than a feature you copied without understanding.
- Design work: sketches of your screens, a data model, a plan for how pieces connect.
- Setting up infrastructure: the repository, the project skeleton, the build, the emulator.
- Writing code that works, obviously. But this is one form of progress among several, not the only one.

Progress does not include

- Nothing at all. "I have been busy" is not progress, and neither is a log with no artifact attached.
- Trivial work that did not move the project. Renaming files or reorganizing folders is maintenance, not progress.
- Plans and promises. What you intend to do next is not something you did.
- A statement that you made no progress. Reporting the absence of work is not a substitute for the work.
- Excuses and explanations for why nothing happened. These may be entirely true, and they still are not progress.

A log without a qualifying artifact is a missed log. It counts exactly the same as not submitting one at all. Given how much counts as an artifact, there is no reason to end up here.

When It Is Not Going Well

Projects stall. Tools misbehave. That is not a problem in itself, and it is not something you will be penalized for.

What matters is what you do about it, and when. If you are stuck, deal with it then: come see me, send an email, ask for help. Do not wait for the next log to mention it, since the log is a report on what already happened, not a request for rescue.

If you need more time, spend a token. That is what tokens are for, and using one is the professional move, not a failure.

Going quiet is the one thing that does not work. A student who comes to me in week nine because the architecture is not working is a student I can help. A student who says nothing until week seventeen is a student I could not help, and by then the semester is gone.

Failing at something is fine. Not dealing with it is not.

8. The Midpoint Video

Due Week 7. Ten to fifteen minutes. This is a technical demonstration of what you have built, not a status report and not a reflection.

Your project is not expected to be complete, and parts of it may not work yet. That is fine. What matters is that you can clearly explain what you built, what you attempted, and why you made the choices you made.

This is a chance to show what you accomplished, not to account for what you did not. The flexibility built into this project, the schedule you designed yourself and the tokens you hold, already gives you room to adjust scope and timeline throughout the semester. Because of that, there is no need to justify a lack of progress in this video.

The video must not include reflections, apologies, excuses, or explanations for why more was not accomplished. A video that is primarily apologies or excuses rather than a demonstration of completed work will be rejected and does not meet the specification. Frame it as an opportunity to show what you have built and take some pride in it.

Three ways to do it

- Record a video and share a link (OneDrive, YouTube, or a similar service)
- Meet with me in person, scheduled in advance
- Meet with me by video conference, scheduled in advance

If you meet in person or by video conference, schedule it far enough ahead that it happens by the deadline. For a live session, what you submit is the date and time we met rather than a link.

Suggested Structure

- **Introduction (about 1 minute):** app name, what it does, who it is for

Spend no more than about a minute introducing the project. Everything else should demonstrate and explain the work you completed in the first seven weeks: what you built, the decisions you made, and why. You may be asked to walk me through your work in person. Go into enough technical depth to show that you genuinely understand what you built, not just that it exists.

- **Demonstration and explanation (9 to 14 minutes):** show what you built: the app running, or a detailed walkthrough of the code and architecture. Explain the decisions behind it and go deep enough to demonstrate real understanding.
- **If you do not yet have application code running:** demonstrate the work that is leading to it: a custom data structure you built, a framework you learned, a technique you had to

work through. Show it and explain it in your own words.

Production quality does not matter. Technical depth and an honest account of what you actually built matters.

9. The Eleven Specifications

Table 4. *The eleven specifications*

#	Specification	Due	Type
1	Proposal submitted and approved	Week 2	Gate
2	Log 1 submitted, with artifact	Week 4	Ladder
3	Log 2 submitted, with artifact	Week 6	Ladder
4	Midpoint video submitted, demonstrating real material	Week 7	Ladder
5	Log 3 submitted, with artifact	Week 9	Ladder
6	Log 4 submitted, with artifact	Week 11	Ladder
7	Log 5 submitted, with artifact	Week 13	Ladder
8	Log 6 submitted, with artifact	Week 16	Ladder
9	Final deliverable and complete repository submitted	Week 17	Gate
10	Final video submitted, including demonstration	Week 17	Gate
11	Final documentation form submitted	Week 17	Gate

10. How Your Project Grade Is Determined

Two things are gates. Everything else sets the grade.

Gate 1: The Proposal

No approved proposal means you are laid off, and there is no project to grade, which is an F.

Gate 2: The Week 17 Submission

All three Week 17 items are required: the final deliverable and repository, the final video, and the documentation form. Missing any one of them is an F, regardless of everything else you did.

The Grade Ladder

If both gates are cleared, your grade is set by the midpoint video and your six logs. A log counts as met if it was submitted on time, reports real progress, and includes a qualifying artifact, or if it was covered by a token.

Table 5. *Grade ladder*

Grade	Points	Requirements
A	100	Midpoint video submitted, and all 6 logs met.
B	85	Midpoint video submitted, and 5 of 6 logs met.

Grade	Points	Requirements
C	75	Midpoint video submitted, and 4 of 6 logs met.
D	60	Midpoint video not submitted; or submitted, but 3 or fewer logs met.
F	0	Any gate not met (no approved proposal, or any Week 17 item missing).

These are the only possible scores for this project. There is no partial credit within a grade. You either met the specifications for that grade or you did not.

The midpoint video is required for a C or higher. Below that, the number of logs you met is what separates a C from a D: four of six logs, with the video, holds the floor at C, and anything less than that puts you at D regardless of the video. You have two tokens and a full week of recovery time on any single item, so there is no real reason to fall short.

Note what this means. A log you handled with a token still counts as met. Communicating and adjusting costs you a token, not a grade. Going silent is the thing that moves you down the ladder.

What Counts as Met

- A log is met when it is submitted on time, reports real progress, and includes a qualifying artifact. All three are required. There is no length requirement, and the artifact is not judged on quality.
- The midpoint video is met when it demonstrates actual project material in sufficient technical depth. A video that is mostly introduction, reflection, or apology does not meet it the requirements.
- The final deliverable is met when your code and complete repository are submitted and correspond to the app you proposed.
- The final video is met when it is given and focuses on demonstrating the project, whether fully completed or not.
- The documentation form is met when it is submitted with its fields completed.

A submission missing the parts a requirement calls for has not been made, and the specification remains unmet. This is a check that the required components are present, not a judgment of how good they are.

11. Tokens

You receive two tokens at the start of the semester. Things come up that nobody plans for, and tokens exist to cover them. Using one is the professional move. It is what you do instead of going quiet.

How tokens work

- Two tokens for the entire semester, and no additional tokens are granted.
- A token may be used on the proposal, any log, or the midpoint video. It is a shared pool, so what you spend early is not available later.
- Tokens cannot be applied to any Week 17 item. The term ends and there is no time to accommodate them.

- A token buys either an extension or one revision and resubmission of that item.
- A log covered by a token counts as met for the grade ladder.

Timing, which is the part that matters

- You must request a token within three days of the original deadline.
- A token moves that item's deadline to seven days after its original due date. This date is fixed.
- Requesting sooner gives you more time to do the work. Requesting later does not move the date; it only leaves you less time.
- A token applies only to the current item. Once the recovery period ends, that item is closed permanently and cannot be reopened later with a token.

Read that again. If you miss a Monday deadline and come to me that same day, you have until Sunday. If you wait until Wednesday, you still have until Sunday. Delay costs you working time, not deadline time.

12. Final Submission and Presentation

Three things are due in Week 17: your deliverable and repository, your final video, and your documentation form. All three are required. Missing any one of them is an F, regardless of everything else you did.

On timing. Tokens cannot be applied to any Week 17 item. The term ends and there is no time to accommodate an extension. Plan accordingly.

Deliverable and Repository

Everything goes in the repository. All of it.

- All source code and the full Android Studio project
- Any databases, assets, and resource files
- All documents, design work, and logs with their artifacts
- All AI conversations, prompts and outputs, as PDFs
- Anything else you produced for this project

Work that does not run is still submitted. Broken code, abandoned attempts, and dead ends all go into the submission. They are evidence of what you did, and this project has never required that everything work.

Organize your repository folder so a reader can find things, and include a README that says what the app is, what works, and how to build or run it.

Final Video

Minimum ten minutes, live or recorded. This is a demonstration of the completed project, not a personal reflection.

Focus entirely on demonstrating what you built. Explain what has been implemented, what remains unfinished if anything does, and the current state of the project. As with the midpoint video, do not include personal reflections or a discussion of what you learned from the experience. The emphasis stays on the product: what it does, how it works, and where it stands.

Requirements

- A demonstration is required, live or recorded.
- Show your code and the app itself, wherever it stands.
- If you present live, test everything first and have a backup recording or screenshots.

Documentation Form

Fill out the provided form. It is a form, not a report, and it is written the same way the video is framed: about the project, not about you. There is no length requirement and no essay.

Its fields cover the project itself: what was completed, how it functions, what state it is in, what future work would look like, the updated capability map, resources used, and AI use. There is no field asking what you learned personally from the experience; the emphasis throughout stays on the product and its status.

13. AI and Outside Resources

This project operates under different terms than the weekly assignments in this course. For the project, you may use generative AI and other outside resources, provided you document what you used and what your role in the result was.

Keep every AI conversation from the start of your project. These are artifacts of your work, and they also count toward the artifact requirement on your logs. Submit them with your logs and in your final submission as PDFs. You are not graded on the content of these conversations.

Be honest about it. There is no reason to claim you used AI only to learn something if it generated your starting code. Documented use is expected and is not penalized. Undisclosed use is a different matter and falls under the academic integrity policy in the syllabus.